

Automated Java Method Naming Task Report

Li Zhen

Experiments & Results: I constructed the dataset using the SEART GitHub search engine. I queried for Java repositories with at least 100 commits, at least 10 contributors, a minimum of 10 stars, and excluded forks, with the search results sorted by number of commits in descending order. I downloaded the list and used a script to clone the top 500 repositories to my local machine. Using javalang, I parsed all Java files and extracted method declarations, stopping mining once I had about 2000k methods. I removed exact duplicates based on the method body, filtered out all constructor methods, filtered out any method whose name contained the string `test` (these were mostly auto-generated or project-specific names like `test\_0967`, `test\_0564`, etc.), and discarded methods longer than 256 tokens by counting code tokens obtained by splitting on whitespace and basic punctuation. In addition, to avoid noisy or potentially sensitive identifiers, I filtered out any method whose name or body contained substrings such as `accesssecret`, `access\_secret`, `accesskey`, `access\_key`, `secretkey`, `secret\_key`, `\_id`, `user\_id`, or `client\_id`. After these cleaning and filtering steps, I capped the dataset at 50k unique methods and then split it into 40k training methods and 10k test methods (80/20 split).

I first trained for one epoch with max sequence length 1124, per-device batch size 1 and gradient accumulation 4 (batch size 4), using AdamW with learning rate $1e-4$; this run had about 9.8k training steps, and a final training loss of about 2.32, yielding an exact-match accuracy of **44.6%** on the 10k-method held-out test set using greedy FIM inference with up to 8 generated tokens. After this initial run, I adjusted the training hyperparameters to learning rate 5×10^{-5} , weight decay 0.01, and warmup ratio 0.05, and retrained for one epoch with the same data and batch settings; this second run reported a final training loss of about 1.82 and achieved a higher exact-match accuracy of **48.84%** on the same 10k-method test set. Finally, I keep the same data and hyperparameters while increasing the number of epochs, I observed smaller but consistent gains: training for two epochs yielded an exact-match accuracy of **51.84%**, and training for three epochs further improved the exact-match accuracy to **52.45%** on the same 10k-method test set.

Analysis: Based on final improved model's prediction result which obtained an exact-match accuracy of 52.45 % (5245 correct, 4755 wrong). Accuracy is clearly length-sensitive: short names (≤ 10 chars) reach 62.16 %, medium names (11–20) drop to 46.12 %, and long names (> 20) to 32.76 %; the same pattern appears by subtoken count, with 62.51 % for 1-token names, 52.59 % for 2–3 tokens, and 36.20 % for names with more than 3 tokens. Subtoken coverage shows that in 99.23 % of all methods every subtoken of the true name appears somewhere in the method body (roughly 99 % for both correct and wrong cases), so the relevant lexical cues are almost always present in the code. For wrong predictions, the average string similarity of 0.389 indicates that many errors are near-misses rather than completely unrelated names. Overall, these metrics suggest the model is strong on short, simple names but struggles to compose longer, multi-token names even when all the necessary subtokens are visible in the method body. I also checked two additional aspects. First, I looked for cases where the only difference between the true and predicted name is capitalization; there are only 15 such examples, which is about 0.32% of all wrong predictions, so simple case-sensitivity is almost never the reason a name is counted as incorrect. Second, I analyzed the length of the method bodies: in my test set they range from 3 to 256 approximate code tokens (average 36.22), and using data-driven cut points at 12 and 32 tokens I grouped them into short (≤ 12 tokens), medium (13–32 tokens) and long (> 32 tokens) methods. The model is most accurate on short methods (about 60% correct), slightly worse on medium methods (about 53% correct), and clearly struggles more on long methods (about 43% correct), which suggests that larger and more complex bodies make precise method naming significantly harder even when the relevant words are present in the code.

Building on this analysis, I looked more closely at the high-similarity “near-miss” errors and saw four main challenges for the model. First, many failures involve arbitrary or project-specific identifiers (case numbers, suffix digits, or internal IDs) where the code does not really contain enough information to choose the exact

index, for example `mcpatcherforge$redirectColor17` vs `mcpatcherforge$redirectColor1`. Second, some errors reflect missing very fine-grained semantic distinctions, such as confusing which numeric or color variant should be chosen, which requires deep understanding of project conventions rather than just local code cues. Third, a few predictions introduce slightly odd or unexpected characters or variants (for instance extra underscores or unusual capitalization), showing that the model sometimes drifts toward locally plausible but stylistically inconsistent tokens. Fourth, for long composite names the model struggles to keep every segment stable, as in `extractNestedBootStrapJar` vs `extractNestedBootstrapJar` or `writeFloat8Vector` vs `writeFloat4Vector`, where it preserves most of the structure but silently alters one subpart (for instance “BootStrap” vs “Bootstrap” or “8” vs “4”). Together, these patterns suggest that the remaining errors are concentrated in very detailed, convention-heavy aspects of naming, rather than in understanding the broad intent of the method.

Based on the “most off” wrong predictions in the new run, I see a similar pattern to before even though all these pairs have zero string similarity. Many pairs still have reasonably high semantic similarity (for example `floorDiv` vs `rad2deg`, `process` vs `getJSON`, `capacity` vs `grow`, `resize` vs `grow`, `hasProperty` vs `contains`, `canInsertBreak` vs `isSimpleExit`, or `month` vs `year`), which shows that the model often stays in the right functional neighbourhood but confuses different lifecycle stages or roles, such as sizing versus growing, checking a property versus testing containment, or different loop control conditions. Several other pairs involve very short and heavily overloaded verbs or nouns (for example `save` vs `load`, `build` vs `getInstanceInfo`, `copy` vs `getChunkData`, `open` vs `wrap`, `add` vs `putToMap`, `end` vs `start`, or `create` vs `getConfig`), where the method body does not clearly privilege one particular naming choice and the model defaults to another common action. There is also a smaller subset of genuinely off cases where both string and semantic similarity are low, such as `fromTag` vs `init` or `setUp` vs `init`, which suggests that for some methods the available lexical and structural cues in the body are either too weak or too noisy for the model to recover the intended concept. Overall, these “most off” errors reinforce that residual mistakes mainly arise for very short or overloaded names and for situations where several semantically close but functionally distinct actions are plausible and the model picks a different one from the developer’s choice.

As I also inspected the wrong predictions more closely, I compared how often subtokens from the true name versus “false-only” predicted subtokens appear in the method body and found that in 79.56% of wrong cases the true subtokens actually occur more frequently, only 15.54% favor the predicted-only subtokens, and 4.90% are ties, suggesting that the model’s mistakes are not simply driven by token frequency in the code but by stronger learned priors over naming patterns. I then analyzed token-set relationships and observed that almost none of the errors are just re-ordering of the same subtokens (0.04%), while 7.61% are “over-specified” (prediction is a superset of the true subtokens), 12.72% are “under-specified” (prediction is a subset of the true subtokens), and the remaining 79.62% are mixed or disjoint sets, indicating that most residual errors involve genuine content mismatches rather than trivial order changes, with a noticeable portion reflecting overly long or overly short but still related names.

I analyzed the subset of wrong predictions that a WordNet-based heuristic classified as “synonym-like”: quantitatively, about 14.30% of all wrong cases (660 out of 4 755) fall into this synonym-like bucket, 0% into an “ambiguous-like” bucket under the same criteria, and the remaining 85.70% into other wrong cases. Within the synonym-like subset I observed six recurring patterns: (i) same operation but slightly different object or scope, such as `getLoginUserInfo` predicted as `getLoginInfo` or `getRedisCacheInfo` predicted as `getCacheInfo`, where the model chooses a closely related but more general or different target entity; (ii) morphological or minor lexical variants, for example `getAssigns` predicted as `getAssignments`, `loginInfoList` predicted as `getLoginInfoList`, or `isVarArg` predicted as `isVarArgs`, which are almost interchangeable from a human perspective but still counted as wrong under exact-match; (iii) symmetric or closely related operations, such as `exitUnitToUnitInterval` predicted as `enterUnitToUnitInterval`, `onChildViewRemoved` predicted as `onChildViewAdded`, or `dropColumnFamily` predicted as `deleteColumnFamily`, where the model clearly understands the conceptual family but sometimes

predicts the “twin” action instead of the reference one; (iv) type or role confusions and small qualifier changes, including cases like `getRelType` predicted as `getRelName`, `buildBasePosDeleteWriter` predicted as `buildPosDeleteWriter`, or `getShaderPath` predicted as `getShaderName`, where the underlying resource is the same but the predicted name refers to a different attribute or a more or less specific variant; (v) genuinely semantic shifts that would be problematic in practice, such as `isPhoneDuplicated` predicted as `isEmailDuplicated`, `getSafeHoles` predicted as `getUnsafeHoles`, `getWarnsCount` predicted as `getErrorsCount`, or `getZ` predicted as `getY`, where the model mixes nearby but meaningfully different concepts (phone vs email, safe vs unsafe, warnings vs errors); and (vi) opaque, ID-like names (for instance `aTaintCase0033` predicted as `aTaintCase0013`), where the model stays in the right family prefix but fails to guess the essentially arbitrary numeric or suffix component. Overall, these metrics and patterns show that the synonym-like bucket captures a non-trivial but still minority portion of the errors where the model’s prediction is not random noise but a semantically related name; within this subset there are two main groups, benign variants (morphology changes, extra “get”, plural vs singular, slightly wider or narrower scope) that would often be acceptable in real code, and genuinely harmful shifts (safe vs unsafe, phone vs email, `NetId` vs `UserId`, warnings vs errors) where the model has learned the right region of the naming space but still picks the wrong facet, suggesting that improving semantic precision remains an important open problem.

Finally, I compared the methods that were previously mispredicted by the 48%-accuracy model but correctly predicted by the 52%-accuracy model, and I observed several consistent improvement patterns from the model. In many cases the new model moves from generic but plausible names to more precise ones (for example, refining `getConfig` into `getApiConfig` or `create` into `createDumpDirectory`), showing better use of contextual cues in the body. It also corrects the polarity or role of operations, more reliably distinguishing “enter” versus “exit”, “create” versus “drop”, or add/remove-style actions based on control-flow and side effects. The improved model adheres more closely to API and framework naming conventions (for example Android callbacks, manager classes, or SQL-style identifiers), and produces cleaner boolean/state predicates by systematically preferring predicate forms like `isRequired` or `isNotEmpty` over bare nouns or verbs. I further see more accurate handling of singular versus plural and collection semantics, where the model now chooses names such as `listOptimizers` or `getServerCatalogs` when the code manipulates collections, indicating sharper discrimination among closely related alternatives rather than settling for a nearby but incorrect sibling name.

Conclusion: In conclusion, the fine-tuned FIM model does learn meaningful naming semantics—its best predictions align well with project vocabulary and code structure—but it still struggles with subtle semantic distinctions (phone vs email, safe vs unsafe), arbitrary ID-like names, and keeping long multi-part names fully consistent. The error analysis shows that many wrong cases are “near misses” (small lexical changes, facet swaps within the same conceptual family) rather than random noise, which suggests that the model is strong at operating in the right naming region but weak at enforcing precise constraints on what is and is not an acceptable name for a specific method.

Future Direction: In the future, I plan to address these limitations with a Tratto-style neuro-symbolic design. Instead of letting the LLM freely generate any token sequence, a symbolic front-end will parse the method, mine verbs, objects and qualifiers from the body and signature, and apply a small naming grammar (for example, [Prefix] Verb [Object] [Qualifier]) to construct a small candidate set of name subtokens drawn from the actual code and project vocabulary. On top of this, I would use a secondary LLM as a ranker that receives richer context: project-level naming statistics and conventions, a few representative caller and callee snippets, and the names of neighbouring methods in the same file or class. Given this context and the grammar-generated candidate names (with simple numeric features such as coverage, length and style flags), the ranker LLM would select or re-rank the most appropriate name for that specific project and usage. This should reduce arbitrary IDs and facet swaps (by constraining available objects and qualifiers), stabilize long composite names (by fixing the

token multiset and structure before ranking), and better enforce global consistency with the surrounding codebase, while keeping the overall system interpretable: the grammar and candidate generation encode explicit naming rules, and the LLM focuses on fine-grained semantic judgment and style alignment that pure static heuristics cannot easily capture.

Screenshots:

Dataset:

```
Total collected methods (before dedup): 200035
Reached max_unique = 50000, stopping dedup.
Total unique methods collected (capped): 50000
Train examples: 40000, Test examples: 10000
```

First training result:

```
{'loss': 1.4922, 'grad_norm': 20.79466465451816, 'learning_rate': 7.382727272727272e-08, 'epoch': 1.0}
{'train_runtime': 43428.8421, 'train_samples_per_second': 0.908, 'train_steps_per_second': 0.227, 'train_loss': 2.3181177932148113, 'epoch': 1.0}
100%|
Training done.
Saved fine-tuned model + tokenizer to qwen_methodname_finetune3
```

Second training result:

```
{'loss': 1.2636, 'grad_norm': 26.3583338482688, 'learning_rate': 3.33843623323648e-08, 'epoch': 1.0}
{'train_runtime': 45130.3599, 'train_samples_per_second': 0.874, 'train_steps_per_second': 0.218, 'train_loss': 1.8209629728218788, 'epoch': 1.0}
100%|
Training done.
Saved fine-tuned model + tokenizer to E:\qwen_methodname_finetune
```

Third training result:

```
{'loss': 0.6155, 'grad_norm': 42.33873748779297, 'learning_rate': 8.274610292547512e-08, 'epoch': 2.0}
{'train_runtime': 10856.1844, 'train_samples_per_second': 7.265, 'train_steps_per_second': 1.816, 'train_loss': 1.3578660262502804, 'epoch': 2.0}
100%|
Training done.
```

Fourth training result:

```
{'train_runtime': 14595.2946, 'train_samples_per_second': 8.222, 'train_steps_per_second': 2.055, 'train_loss': 1.0725995775858561, 'epoch': 3.0}
100%|
Training done.
Saved fine-tuned model + tokenizer to E:\qwen_methodname_finetune3
```

First test result:

```
W1117 01:18:25.378000 13004 .venv\Lib\site-packages\torch\dl
Loaded 10000 test examples.
The attention mask is not set and cannot be inferred from in
Exact-match accuracy on test set: 44.62% (4462/10000)
(qwen) PS D:\methodname_prediction>
```

Second test result:

```
Loaded 10000 test examples.
The attention mask is not set and cannot be inferred from
Exact-match accuracy on test set: 48.84% (4884/10000)
Saved all predictions to predictions2.jsonl
```

Third test result:

```
The attention mask is not set and cannot be inferred from :
Exact-match accuracy on test set: 51.84% (5184/10000)
Saved all predictions to predictions3.jsonl
```

Fourth test result:

```
Loaded 10000 test examples.
The attention mask is not set and cannot be inferred fr
Exact-match accuracy on test set: 52.45% (5245/10000)
Saved all predictions to predictions5.jsonl
```

Analysis of the final model's results:

```
Total evaluated examples: 10000
Correct: 5245, Wrong: 4755
Accuracy: 52.45%

Accuracy by true method name length:
  short (<=10 chars): 62.16% (4717 examples)
  medium (11-20 chars): 46.12% (4358 examples)
  long (>20 chars): 32.76% (925 examples)

Method body length statistics (in approximate code tokens):
  min: 3, max: 256, average: 36.22, cut points: <= 12, 13-32, > 32

Accuracy by method body length:
  short bodies (<= 12 tokens): 59.78% (2140 correct, 1440 wrong, 3580 examples)
  medium bodies (13-32 tokens): 53.14% (1658 correct, 1471 wrong, 3139 examples)
  long bodies (> 32 tokens): 45.88% (1437 correct, 1844 wrong, 3281 examples)

Accuracy by number of subtokens in true name:
  1 token: 62.51% (2286 examples)
  2-3 tokens: 52.59% (6374 examples)
  >3 tokens: 36.28% (1420 examples)

Subtoken coverage of TRUE name in method body (all examples):
  all subtokens appear: 9923 (99.23%)
  some subtokens appear: 0 (0.00%)
  no subtokens appear: 77 (0.77%)

For CORRECT predictions:
  total: 5245
  all subtokens appear: 5192 (98.99%)
  some subtokens appear: 0 (0.00%)
  no subtokens appear: 53 (1.01%)

For WRONG predictions:
  total: 4755
  all subtokens appear: 4731 (99.50%)
  some subtokens appear: 0 (0.00%)
  no subtokens appear: 24 (0.50%)

Case-only wrong predictions (capitalization differences only):
  15 (0.32% of all wrong predictions)

Average similarity for wrong predictions (0-1): 0.389

Top 5 near-miss wrong predictions (high similarity):
-----
True: buildNegativeViewUsersQueryConstraint | Pred: buildNegativeViewUserQueryConstraint | sim = 0.973
True: mcpatcherforge$redirectColor17 | Pred: mcpatcherforge$redirectColor1 | sim = 0.967
True: setDoubleRangeObject2DoubleMap | Pred: setdoubleRangeObject2DoubleMap | sim = 0.967
True: firstLetterOfWordCapitalised | Pred: firstLetterOfWordsCapitalised | sim = 0.966
True: onMonitorContentedEntered | Pred: onMonitor ContendedEntered | sim = 0.962
-----
```

```
Top 5 most off wrong predictions (low similarity):
-----
True: floorDiv | Pred: rad2deg | string_sim = 0.000 | semantic_sim = 0.000
True: after | Pred: clean | string_sim = 0.000 | semantic_sim = 0.500
True: buildPath | Pred: resolved | string_sim = 0.000 | semantic_sim = 0.311
True: refresh | Pred: apply | string_sim = 0.000 | semantic_sim = 0.400
True: copy | Pred: getChunkData | string_sim = 0.000 | semantic_sim = 0.400
-----

Semantic-style breakdown for WRONG predictions (symmetric subtokens):
  synonym-like (high similarity, overlap, and some WordNet synonym subtokens): 680 (14.38%)
  ambiguous-like (high similarity and overlap, but no synonym pair): 0 (0.00%)
  other wrong cases: 4075 (85.70%)

Example synonym-like WRONG predictions (WordNet):
-----
True: handleBindException | Pred: handleMethodException
True: generateWithVarargsKeys | Pred: generateWithMapKeys
True: enterErrorIdent | Pred: exitErrorIdent
True: atIntCase0033 | Pred: atIntCase0013
True: asStructlikeKey | Pred: asStructlike
-----

For WRONG predictions: comparison of subtoken occurrences in method body
  more FALSE-only subtokens than TRUE subtokens: 739 (15.54%)
  more TRUE subtokens than FALSE-only subtokens: 3783 (79.56%)
  equal or both zero: 233 (4.90%)

For WRONG predictions: token order and subset/superset patterns
  same tokens but different order: 2 (0.04%)
  TRUE tokens subset of PRED tokens (over-specified prediction): 362 (7.61%)
  PRED tokens subset of TRUE tokens (under-specified prediction): 605 (12.72%)
  mixed / disjoint token sets: 3786 (79.62%)

Totally wrong predictions (no overlapping subtokens and no WordNet synonym pair):
  count: 1400 (29.44% of all wrong predictions)

Example totally wrong predictions:
-----
True: init | Pred: unbind | string_sim = 0.333 | semantic_sim = 0.000
True: clone | Pred: getTableLoader | string_sim = 0.214 | semantic_sim = 0.667
True: updateDeps | Pred: loopInAnonymousInline | string_sim = 0.095 | semantic_sim = 0.429
True: stop | Pred: dispose | string_sim = 0.286 | semantic_sim = 0.400
True: floorDiv | Pred: rad2deg | string_sim = 0.000 | semantic_sim = 0.000
-----
```

